

1 Executive Summary

This module compares body rate estimates from an IMU and a star tracker. If the two rates differ by more than a configurable threshold for a configurable number of consecutive updates, the module reports a fault and outputs the IMU rate. Otherwise it outputs the star tracker rate. Once a fault is declared it is sticky and persists until the module is reset.

2 Message Connection Descriptions

The following table lists all the module input and output messages. The module msg variable name is set by the user from python. The msg type contains a link to the message structure definition, while the description provides information on what this message is used for.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
imuSensorBodyInMsg	IMUSensorBodyMsgF32Payload	IMU body rate input message (angular rate in body frame)
stBodyInMsg	STAttMsgPayload	star tracker attitude input message (uses <code>omega_BN_B</code>)
navAttMsg	NavAttMsgPayload	navigation output message containing the selected body rate
rateFaultMsg	BodyRateFaultMsgPayload	fault output message indicating IMU and star tracker miscompare

3 Module Parameters

The following table lists all the module parameters that can be set.

Table 2: Module Parameters

Parameter Name	Type	Units	Default	Description	Bounds
bodyRateThreshold	float	[rad/s]	1.0	Euclidean norm threshold for rate miscompare detection	Must be strictly positive (checked in setter)
faultPersistenceLimit	uint32_t	[-]	1	Number of consecutive threshold violations required to declare a fault	Must be ≥ 1 (checked in setter)

4 Algorithm Input/Output

The following tables list the inputs and outputs of the pure algorithm `update()` method, independent of the Xmera messaging layer.

Table 3: Algorithm Inputs

Variable	Type	Units	Description
imuOmega_BN_B	<code>Eigen::Vector3f</code>	[rad/s]	IMU body rate in body-frame components

Variable	Type	Units	Description
stOmega_BN_B	Eigen::Vector3f	[rad/s]	Star tracker body rate in body-frame components

Table 4: Algorithm Outputs (BodyRateMiscompareOutput)

Variable	Type	Units	Description
omega_BN_B	Eigen::Vector3f	[rad/s]	Selected body rate (IMU if fault, star tracker otherwise)
bodyRateFaultDetected	bool	[-]	True if fault persistence limit has been reached

5 Algorithm Description

The body rate miscompare algorithm operates on two input vectors, the IMU body rate and the star tracker body rate, both expressed in body-frame components and in consistent units (rad/s). Let the two inputs be ω_{imu} and ω_{st} .

The algorithm computes the difference

$$\Delta\omega = \omega_{st} - \omega_{imu}$$

and compares its Euclidean norm to a positive threshold τ :

$$\|\Delta\omega\| > \tau$$

If the threshold is exceeded, a persistence counter is incremented. If the threshold is not exceeded, the counter resets to zero. When the persistence counter reaches the configurable fault persistence limit N , the algorithm declares a fault and outputs the IMU rate. Otherwise, it outputs the star tracker rate. The output also includes a boolean flag indicating whether the fault was detected.

Once a fault has been declared, it is **sticky**: the algorithm continues to output the IMU rate and report a fault on all subsequent calls, regardless of the input values. The persistence counter is no longer evaluated after the fault becomes sticky.

6 Algorithm Assumptions and Limitations

- The threshold τ must be strictly positive. A zero or negative threshold is invalid.
- The fault persistence limit N must be at least 1.
- The two input rates must be expressed in the same frame and units.
- The comparison uses a strict greater-than check ($\|\Delta\omega\| > \tau$).
- The fault is sticky: once declared, it cannot be cleared without resetting the algorithm.
- The algorithm does not validate the physical plausibility of the input rates beyond finite arithmetic.

7 Module Description (Xmera Usage)

The BodyRateMiscompare simulation module provides the Xmera integration layer for the algorithm. It:

- Reads imuSensorBodyInMsg (IMU rate) and stBodyInMsg (star tracker attitude message).
- Converts the star tracker payload to float and extracts omega_BN_B.

- Calls the body rate miscompare algorithm to select the output rate.
- Writes navAttMsg with the selected body rate and rateFaultMsg with the fault flag.

The module sets the output time tag using the simulation callTime.

8 Module Assumptions and Limitations

- Both input messages must be linked before reset. The module throws an error if either input is not connected.
- The module only uses the body rate from the star tracker message and ignores other attitude fields.
- The module does not perform unit conversions beyond the internal float conversion for the star tracker message.

9 User Guide

Typical usage in Python is:

```
module = bodyRateMiscompareF32.BodyRateMiscompare()
module.modelTag = "bodyRateMiscompare"
module.bodyRateThreshold = body_rate_threshold_rad_per_sec
module.faultPersistenceLimit = 3 # require 3 consecutive violations before declaring fault

module.imuSensorBodyInMsg.subscribeTo(imu_msg)
module.stBodyInMsg.subscribeTo(st_msg)
```

The output body rate is available on navAttMsg, and the fault status is available on rateFaultMsg.

The `faultPersistenceLimit` parameter (default 1) controls how many consecutive threshold violations are required before the fault is declared. Setting it to 1 means the fault triggers on the first violation.